目录

图论

最小生成树 - Prim 算法 (堆优化版)

算法说明

用途：求无向图的最小生成树 (MST) 的权值之和。

时间复杂度：O(ElogE) 或 O(ElogV)。

适用场景：适合稠密图。对于稀疏图，Kruskal 算法通常代码更短，但堆优化的 Prim 性能也十分优秀。

最小生成树 prim.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 5050, M = 2e5 + 10;
const int INF = 0x3f3f3f3f;

// 链式前向星存图
struct E {
    int v, w, x;
} e[M * 2]; // 无向图记得开两倍空间

int n, m, h[N], cnte;
int dis[N];     // 存储点到当前生成树集合的最小距离
bool vis[N];    // 标记点是否已加入生成树
int res = 0;    // 最小生成树权值和
int cnt = 0;    // 已加入生成树的点数量

void adde(int u, int v, int w) {
    e[++cnte] = E{ v, w, h[u] };
    h[u] = cnte;
}

struct S {
    int u, d;
    // 优先队列小根堆，距离小的优先
    bool operator<(const S& x) const {
        return d > x.d;
    }
};

priority_queue<S> q;

// 返回 true 表示成功生成 MST，false 表示图不连通
bool Prim() {
    // 初始化
    memset(dis, 0x3f, sizeof(dis));
    memset(vis, 0, sizeof(vis));
    res = 0;
    cnt = 0;

    // 从 1 号点开始
    dis[1] = 0;
    q.push({ 1, 0 });

    while(!q.empty()) {
        // 优化：如果已经选了 n 个点，说明生成树构建完成
        if(cnt >= n) break;

        int u = q.top().u;
        int d = q.top().d;
        q.pop();

        if(vis[u]) continue;
        vis[u] = 1;
        res += d;
        cnt++;

        for(int i = h[u]; i; i = e[i].x) {
            int v = e[i].v;
            int w = e[i].w;
            // 如果通过 u 到达 v 的距离比 v 当前记录的距离更短
            if(w < dis[v]) {
                dis[v] = w;
                q.push({ v, w });
            }
        }
    }

    // 判断是否所有点都在生成树内
    return cnt == n;
}

/* 使用示例：
 cin >> n >> m;
 for(int i=1; i<=m; i++){
     int u,v,w; cin>>u>>v>>w;
     adde(u,v,w); adde(v,u,w);
 }
 if(Prim()) cout << res << endl;
 else cout << "impossible" << endl;
*/
```

数学

数论 - 线性筛质数 (欧拉筛)

算法说明

* **用途**：快速筛选出 $1 \sim n$ 范围内的所有质数。
* **时间复杂度**：$O(n)$。
* **适用场景**：适用于 n 较大（如 10^7 级别）的情况。相比埃氏筛，它保证每个合数只被其**最小质因子**筛去一次，因此是线性的，效率极高。

质数筛.cpp

```cpp
const int N = 1e7 + 10;
int pri[N], cnt;
bool st[N];
void getpri(int n)
```

```
{
    for(int i = 2;i <= n;i++)
    {
        if(!st[i])pri[cnt++] = i;
        for(int j = 0;pri[j] <= n / i;j++)
        {
            st[pri[j] * i] = 1;
            if(i % pri[j] == 0)break;
        }
    }
}
```

数据结构

图论 - 最近公共祖先 (LCA) - 倍增法

算法说明

* **用途**：在有根树中，快速查询两个节点的最近公共祖先。
* **时间复杂度**：
* 预处理：$O(N \log N)$。
* 单次查询：$O(\log N)$。
* **适用场景**：适用于 N, M 较大（如 $10^5 \sim 10^6$）且需要在线查询的情况。
* **注意事项**：
1. `f` 数组第二维的大小由 N 决定，通常 $2^{19} > 5 \times 10^5$，所以开 20 足够。
2. 利用 `de[0] = 0`（全局变量默认值）作为哨兵，防止倍增跳越界。

树-LCA 倍增算法.cpp

```
#include<bits/stdc++.h>
using namespace std;

const int N = 500010;

vector<int> g[N];

int f[N][20];  // f[u][i] 表示 u 的第 2^i 个祖先
int de[N]; //深度数组，0号点深度为0作为哨兵
/**
 * DFS 预处理倍增数组和深度
 * @param u 当前节点
 * @param fa 父节点（根节点调用时 fa 传 0）
 */
void dfs(int u, int fa) {
    de[u] = de[fa] + 1;
    f[u][0] = fa;

    // 倍增核心逻辑
    for(int i = 1; i < 20; i++) {
        f[u][i] = f[f[u][i - 1]][i - 1];
    }
    for(int v : g[u])
    {
        if(v == fa) continue;
        dfs(v, u);
    }
}

/**
 * 查询 LCA
 */
int lca(int a, int b) {
    if(de[a] < de[b]) swap(a, b);
    // 1. 将 a 跳到和 b 同一深度
    for(int i = 19; i >= 0; i--) {
        if(de[f[a][i]] >= de[b]) a = f[a][i];
    }
    if(a == b) return a;
    // 2. a 和 b 一起向上跳，直到父亲相同
    for(int i = 19; i >= 0; i--) {
        if(f[a][i] != f[b][i]) {
            a = f[a][i];
            b = f[b][i];
        }
    }
    return f[a][0]; // 返回父节点
}
```